SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4305

Name: M. Ramya sri

Reg. No: 192372144

Departement : CSE(AI)

AT01-C01-ASSESSMENT : CHAPTER 1

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and

XML, and apply Internet protocols and HTTP communication mechanisms for web content

delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

Code of Conduct:

I **_M.Ramya sri (192372144)**_____(Name / Reg No) certify that this submission is my original

work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary

action.

Signature of the student: **M.Ramya sri**

RUBRICS FOR CALCULATION:

Criteria	Weig htage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconception s	Poor understandin g; major misconceptio ns
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

QUESTION AND ANSWERS

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

ANSWER:

1. Identify appropriate HTML5 semantic elements missing in the structure:

2. <header> – Displays the page heading.
3. <main> – Contains the main content.
4. <section> – Groups the registration form.
5. <article> – Holds independent content (optional).
6. <footer> – Displays copyright or contact information.
7. <label> – Provides labels for input fields.

2. Modify the structure to make it standards-compliant:

<form action="/register" method="POST">

Name: <input type="text" name="name">

Email: <input type="email" name="email">

<button>Submit</button>

</form>

3. **Interpret the HTTP request generated during form submission**

When the user clicks Submit, the browser sends an HTTP POST request to the server.

Example:

POST /register HTTP/1.1

Host: example.com

Content-Type: application/x-www-form-urlencoded

name=Rahul&email=rahul@gmail.com

Explanation:

- **POST** → Sends form data to the server.
- **/register** → URL where data is sent.
- **Host** → Server address.
- **Content-Type** → Format of submitted data.
- **Body** → Contains the user's name and email.

4. Analyze the HTTP response cycle from server to browser

1. User fills the registration form.
2. Browser sends an HTTP POST request.
3. Server receives and validates the data.
4. Server stores the registration details in the database.
5. Server sends an HTTP response.

Example Response:

HTTP/1.1 200 OK

Content-Type: text/html

Registration Successful

5. Suggest improvements for usability and accessibility

- Add **labels** for all input fields.
- Use the **required** attribute for mandatory fields.
- Provide clear error messages.
- Use semantic HTML5 elements (header, main, section, footer).
- Ensure keyboard accessibility.
- Add placeholder text (optional).
- Use proper color contrast for readability.
- Validate email format using type="email".

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

ANSWER:

1. Identify improper nesting and missing closing tags

Errors:

- <h1> is not closed.

- is not closed.
- Missing <!DOCTYPE html> declaration.

2. Analyze how different browsers interpret it

- Browsers automatically close missing tags.
- Different browsers may display the page differently.
- Layout may become inconsistent.

3. Identify missing semantic elements

Missing elements:

- <header>
- <main>
- <section>

4. Corrected W3C-Compliant HTML

<!DOCTYPE html>

<html>

<body>

<header><h1>Welcome</h1></header>

About us

</body>

</html>

5. Techniques to ensure cross-browser compatibility

- Use <!DOCTYPE html>.
- Close all HTML tags.
- Use semantic HTML5 elements.
- Validate HTML using the W3C Validator.
- Test the website in multiple browsers.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

ANSWER:

1. Identify issues in the form configuration

- The form uses the **GET** method, which is not suitable for sending passwords because the data appears in the URL.
- The button type is set to **button**, so it does not submit the form.
- For login forms, **POST** and a **submit** button should be used.

2. Analyze why the HTTP request is not triggered

- The HTTP request is not sent because the button type is **button** instead of **submit**.
- A normal button only performs actions through JavaScript.
- Without JavaScript or a submit button, the form is never submitted to the server.

3. Interpret the role of GET method in this scenario

- The **GET** method sends form data through the URL as query parameters.
- It is mainly used to retrieve information from the server.
- Since passwords become visible in the URL, GET is not recommended for login forms.

4. Corrected Implementation

```
<form action="/submit" method="POST">
<input type="text" name="username">
<input type="password" name="password">
<button type="submit">Login</button>
</form>
```

5. Suggest improvements for secure data transmission

- Use the **POST** method instead of GET to hide form data from the URL.
- Enable **HTTPS** to encrypt data during transmission.
- Validate user input and store passwords securely using encryption . This improves both security and user privacy.